PHANTOM PROTOCOL

Technical Specification № 1

Hidden Services

Rendezvous Protocol · .phantom Address Derivation · Introduction Points

March 2026 · Draft v0.1

ABSTRACT

This specification defines Phantom Hidden Services: the mechanism by which a server can accept connections from Phantom Protocol clients without either party learning the other's IP address, and without any third party being able to identify, block, or surveil the server's existence. The specification covers: (1) .phantom address derivation from post-quantum key material and its self-authenticating properties; (2) the introduction point protocol by which a hidden service registers its reachability inside the Phantom mix network; (3) the rendezvous protocol by which client and server establish a shared session key through an intermediary node that learns neither endpoint's identity; (4) the full cryptographic key exchange over this rendezvous channel using a hybrid classical and post-quantum construction; (5) epoch-based descriptor rotation ensuring forward secrecy for hidden service locations; and (6) denial-of-service resistance through proof-of-work client puzzles at the introduction layer. Together these components allow any Phantom node to host a globally reachable, anonymous service with no DNS registration, no hosting provider, no certificate authority, and no identifiable infrastructure.

Table of Contents

1. Problem Statement — What Tor's Hidden Services Get Wrong
2. Design Goals and Security Properties
3. .phantom Address Derivation
4. Hidden Service Key Material
5. Introduction Points
6. Service Descriptor — Format and Publication
7. Client-Side Lookup
8. Rendezvous Protocol — Full Exchange
9. Cryptographic Key Exchange over the Rendezvous Channel
10. Session Establishment and Ongoing Communication
11. Epoch Rotation and Forward Secrecy
12. Denial-of-Service Resistance
13. Rust Implementation Guide
14. Attack Analysis
15. Open Problems
16. References

1. Problem Statement — What Tor's Hidden Services Get Wrong

Tor's onion services (.onion) are the closest precedent to Phantom Hidden Services. They provide a mechanism for servers to accept connections without exposing their IP address, using a rendezvous protocol through introduction points. However, three fundamental design limitations make them insufficient for the Phantom threat model.

1.1 Address Derivation Depends on Classical Cryptography

Tor v3 onion addresses are derived from an Ed25519 public key. An adversary with a cryptanalytically relevant quantum computer can compute the corresponding private key from the public address, allowing impersonation of any hidden service whose address is known. Since onion addresses are publicly distributed — shared in links, listed in directories — a harvest-now-attack-later strategy is viable today. Phantom Hidden Services must be secure against quantum adversaries from inception.

1.2 Introduction Points are Identifiable Tor Relays

Tor's introduction points are ordinary Tor relays listed in the Tor directory. An adversary with access to the directory (which is public) can identify which relays are serving as introduction points for a given hidden service, then mount a targeted attack — compromising or denying service to those specific relays. In Phantom Protocol, introduction points are anonymous Phantom nodes discoverable only via the DHT, providing no identifiable attack surface at the directory level.

1.3 No Verifiable Mixing of Rendezvous Traffic

In Tor, traffic to and from a hidden service passes through relay nodes that are trusted but unverified. A compromised relay on the rendezvous path can observe or manipulate traffic without detection. Phantom Protocol's ZK shuffle proof (Technical Specification № 2) ensures that every relay on the rendezvous path proves correct packet processing — the same guarantee that applies to all Phantom traffic applies equally to hidden service traffic.

The Requirement

Phantom Hidden Services must provide: a self-authenticating address derivable from post-quantum key material; a registration mechanism with no identifiable infrastructure; a rendezvous protocol resistant to quantum adversaries; full verifiable mix-network properties on all rendezvous traffic; and epoch-based rotation ensuring that compromise of current key material does not compromise past or future service locations.

2. Design Goals and Security Properties

Property	Requirement	Mechanism
Server anonymity	Server IP never revealed to client, relays, or observers	Rendezvous via anonymous Phantom node; Sphinx⁺ routing
Client anonymity	Client IP never revealed to server, relays, or observers	Client builds independent Sphinx⁺ circuit to rendezvous node
Self-authenticating address	Client verifies server identity from address alone	Address = H(pk_ed ‖ pk_kyber); no CA required
Post-quantum security	Address and key exchange secure against quantum adversary	Kyber-1024 + Dilithium-3 in address derivation and KEM
Forward secrecy	Compromise of current keys does not expose past traffic	Epoch-based KEM key rotation; ephemeral session keys
DoS resistance	Server cannot be trivially flooded via introduction points	Client PoW puzzle at introduction layer; rate limiting
No identifiable infrastructure	No DNS, no hosting provider, no registrar	DHT-published service descriptors; .phantom TLD is Phantom-internal
Verifiable mixing	Rendezvous path mixing is cryptographically proven	Same ZK shuffle proof as all Phantom traffic (Spec № 2)
Unlinkability	Multiple connections to same service are unlinkable	Fresh rendezvous node per session; ephemeral key exchange

Table 1 — Design goals and their mechanisms

3. .phantom Address Derivation

A Phantom hidden service address is a self-authenticating identifier derived entirely from the service's long-term cryptographic key material. No external registration is required. Any client that possesses a .phantom address can verify the server's identity without consulting any third party.

3.1 Key Generation

A hidden service generates two long-term key pairs at inception. These keys are permanent — they define the service's identity. KEM keys rotate each epoch (see §11); signing keys do not.

```
// Long-term signing keys (permanent — define service identity)
let sk_ed  = Ed25519::generate(&mut rng);
let sk_dil = Dilithium3::generate(&mut rng);

// Long-term KEM keys (rotate each epoch for forward secrecy)
let sk_x25   = X25519::generate(&mut rng);
let sk_kyber = Kyber1024::generate(&mut rng);

// Derive service address (permanent, self-certifying)
let address_bytes = blake3(
    b"phantom-hs-v1"
    || pk_ed.as_bytes()       // 32 bytes
    || pk_dil.as_bytes()      // 1952 bytes
    || pk_x25519.as_bytes()   // 32 bytes  (epoch-0 KEM key, for address derivation
only)
    || pk_kyber.as_bytes()    // 1568 bytes (epoch-0 KEM key, for address
derivation only)
);
// address_bytes: 32 bytes

// Human-readable address
let address = base32_encode(address_bytes) + ".phantom";
// Example: 3xk7m2nqvfgpqrstuvwxyz234567aaaa.phantom
```

Listing 1 — Hidden service address derivation

3.2 Address Format

A .phantom address is the Base32 encoding (RFC 4648, lowercase, no padding) of the 32-byte address_bytes, followed by the ".phantom" pseudo-TLD. The address is exactly 52 characters plus the 8-character ".phantom" suffix: 60 characters total.

```
// .phantom address structure
//  <52 chars base32>  .  p h a n t o m
//  |-- 32 bytes ---|     |---8 chars--|
//
// Example:
//   3xk7m2nqvfgpqrstuvwxyz234567aaaa.phantom
//
// NOT a real DNS name. Resolved entirely within Phantom Protocol.
// No ICANN registration. No DNS resolver consulted.
```

3.3 Self-Authentication

When a client connects to a .phantom address, it verifies that the server holds the private keys corresponding to the address before any application data is exchanged. This prevents impersonation without requiring any certificate authority or PKI lookup.

```
// Client verification at connection time:
VERIFY_SERVICE(address, service_descriptor):
  1. Decode address_bytes = base32_decode(address.strip_suffix(".phantom"))
  2. Recompute: expected = BLAKE3("phantom-hs-v1"
                              || descriptor.pk_ed
                              || descriptor.pk_dil
                              || descriptor.pk_x25519_epoch0
                              || descriptor.pk_kyber_epoch0)
  3. Assert: expected == address_bytes
  4. Verify: Ed25519.Verify(pk_ed, descriptor.canonical_bytes(), descriptor.sig_ed)
  5. Verify: Dilithium3.Verify(pk_dil, descriptor.canonical_bytes(),
descriptor.sig_dil)
  If all 5 checks pass: server identity confirmed. No authority consulted.
```

Listing 2 — Client-side service address verification

4. Hidden Service Key Material

4.1 Key Hierarchy

A hidden service maintains two categories of key material with different lifetimes and rotation policies:

Key	Type	Lifetime	Purpose
sk_ed / pk_ed	Ed25519	Permanent	Service identity signing; used in address derivation and descriptor signing
sk_dil / pk_dil	Dilithium-3	Permanent	Post-quantum identity signing; used in address derivation and descriptor signing
sk_x25519 / pk_x25519	X25519	Per-epoch (1 hour)	Classical KEM for rendezvous key exchange
sk_kyber / pk_kyber	Kyber-1024	Per-epoch (1 hour)	Post-quantum KEM for rendezvous key exchange
sk_intro[i] / pk_intro[i]	Ed25519	Per introduction point session	Authentication of introduction point registration messages

Table 2 — Hidden service key material and lifetimes

4.2 Key Storage and Isolation

Long-term signing keys (Ed25519, Dilithium-3) must be stored separately from the process handling network connections. Compromise of the network-facing process must not expose the signing keys. The recommended pattern is to run a separate key-signing daemon that receives descriptor bytes and returns signatures, with the signing keys never loaded into the same process as the Sphinx⁺ packet handler.

```
phantom-hidden-service/
├── key-daemon/          // holds sk_ed, sk_dil; network-isolated
│   └── signer.rs        // receives canonical bytes, returns signatures
├── network/             // handles Sphinx+ packets; no signing keys
│   ├── intro.rs         // introduction point registration
│   └── rendezvous.rs    // rendezvous protocol handler
└── descriptor/          // assembles and publishes service descriptors
    └── publish.rs
```

Listing 3 — Recommended key isolation process structure

5. Introduction Points

Introduction points are ordinary Phantom relay nodes that have agreed to forward contact requests to a hidden service. They do not know the hidden service's IP address, identity, or location — they know only a per-session introduction key that the service uses to authenticate registration messages.

5.1 Introduction Point Selection

Each epoch, the hidden service selects N_INTRO introduction points from the active Phantom node directory (fetched via the DHT, Technical Specification № 3). Selection criteria:

- Must be Full-tier nodes (bandwidth ≥ 100 Mbps; see NodeDescriptor.capacity_tier).
- Must be at mix_layer 0 or 1 (guard or middle layer). Exit nodes are ineligible.
- Must have uptime_score ≥ 180 (of 255) in the current epoch.
- Must not share an autonomous system (AS) with any other selected introduction point — AS diversity prevents a single ISP compromise from eliminating all introduction points.
- **N_INTRO = 5** by default; configurable between 3 and 10.

```
const N_INTRO: usize = 5;
const MIN_UPTIME_SCORE: u8 = 180;

fn select_intro_points(nodes: &[NodeDescriptor]) -> Vec<NodeDescriptor> {
    let candidates: Vec<_> = nodes.iter()
        .filter(|n| n.capacity_tier == CapacityTier::Full)
        .filter(|n| n.mix_layer <= 1)
        .filter(|n| n.uptime_score >= MIN_UPTIME_SCORE)
        .collect();

    // Select with AS diversity
    let mut selected = Vec::new();
    let mut seen_asns = HashSet::new();
    for node in candidates.choose_multiple(&mut rng, candidates.len()) {
        let asn = ip_to_asn(node.quic_addr.ip());
        if !seen_asns.contains(&asn) {
            selected.push(node.clone());
            seen_asns.insert(asn);
            if selected.len() == N_INTRO { break; }
        }
    }
    selected
}
```

Listing 4 — Introduction point selection with AS diversity

5.2 Introduction Point Registration

The hidden service registers with each introduction point by sending a signed registration message via a Sphinx⁺ circuit. The circuit terminates at the introduction point, not beyond — the introduction point is the exit node of this particular circuit, so it never learns the service's onward path.

```
#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]
pub struct IntroRegistration {
    pub intro_node_id:   [u8; 32],    // the introduction point's node_id
    pub service_address: [u8; 32],    // 32-byte .phantom address
    pub pk_intro_session:[u8; 32],    // ephemeral Ed25519 pubkey for this intro
session
    pub epoch:           u32,
    pub pow_nonce:       [u8; 8],     // DoS resistance (see §12)
    pub sig_ed:          [u8; 64],    // Ed25519(sk_ed, canonical_bytes)
    pub sig_dil:         [u8; 3293],  // Dilithium-3(sk_dil, canonical_bytes)
}

// Registration is sent as the payload of a Sphinx+ FORWARD packet.
// The introduction point decrypts the payload, verifies the signature
// against the service_address (which embeds pk_ed in its derivation),
// and stores the mapping: service_address → pk_intro_session.
```

Listing 5 — Introduction point registration message format

5.3 Introduction Point State

After successful registration, an introduction point maintains the following in-memory state for each registered service:

```
pub struct IntroState {
    pub service_address:  [u8; 32],     // identifies the service
    pub pk_intro_session: [u8; 32],     // ephemeral Ed25519 pubkey for this
session
    pub registered_at:    Instant,
    pub epoch:            u32,
    pub request_count:    u32,           // for rate limiting
    pub pow_difficulty:   u8,            // adaptive DoS resistance
}

// Introduction points do NOT store:
//   - The service's IP address (never known)
//   - The service's return circuit (one-time use, discarded after forward)
//   - Any linkage between different sessions for the same service
```

Listing 6 — Introduction point in-memory state

6. Service Descriptor — Format and Publication

The service descriptor is the record that a client retrieves from the DHT to learn how to contact a hidden service. It contains the service's current KEM public keys (for the rendezvous key exchange) and the list of introduction points. It is signed by the service's permanent signing keys and published to the DHT at the start of each epoch.

6.1 Descriptor Structure

```
#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]
pub struct ServiceDescriptor {
    // ── Identity ──────────────────────────────────────────────
    pub service_address: [u8; 32],     //
BLAKE3(pk_ed||pk_dil||pk_x25519_e0||pk_kyber_e0)
    pub pk_ed25519:      [u8; 32],     // permanent Ed25519 signing pubkey
    pub pk_dilithium:    [u8; 1952],   // permanent Dilithium-3 signing pubkey

    // ── Epoch KEM keys (rotate every hour) ────────────────────
    pub pk_x25519:       [u8; 32],     // X25519 KEM pubkey for this epoch
    pub pk_kyber:        [u8; 1568],   // Kyber-1024 KEM pubkey for this epoch

    // ── Introduction points ───────────────────────────────────
    pub intro_points:    Vec<IntroPointRef>,  // N_INTRO entries

    // ── Epoch ─────────────────────────────────────────────────
    pub epoch:           u32,
    pub published_at:    u64,          // Unix timestamp
    pub expires_at:      u64,          // published_at + 3600 seconds

    // ── DoS parameters ────────────────────────────────────────
    pub intro_pow_bits:  u8,           // required PoW difficulty for intro
requests

    // ── Signatures ────────────────────────────────────────────
    pub sig_ed25519:     [u8; 64],     // Ed25519 over canonical_bytes()
    pub sig_dilithium:   [u8; 3293],   // Dilithium-3 over canonical_bytes()
}

#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]
pub struct IntroPointRef {
    pub node_id:          [u8; 32],   // introduction point's node_id (DHT lookup
key)
    pub pk_intro_session: [u8; 32],   // ephemeral Ed25519 pubkey for this intro
session
}
// Note: no IP address. Client resolves node_id via DHT to get IP.
```

Listing 7 — ServiceDescriptor structure

6.2 DHT Publication

The descriptor is published under a DHT key in the Hidden Service namespace (defined in Technical Specification № 3, Table 4):

```
// DHT key for service descriptor:
dht_key = BLAKE3("phantom-hs-desc-v1" || service_address || epoch.to_be_bytes())
```

```
// Publication:
DHT.store(Namespace::HiddenSvc, dht_key, descriptor.encode())

// Client lookup:
let raw = DHT.fetch(Namespace::HiddenSvc, dht_key).await?;
let desc = ServiceDescriptor::decode(raw)?;
desc.verify(service_address)?;  // self-authentication check
```

6.3 Descriptor Confidentiality

The service descriptor is published in plaintext on the DHT. Any Phantom node — or any adversary with DHT access — can retrieve it. This is intentional: the descriptor contains no information beyond what a connecting client needs (introduction point node_ids and epoch KEM keys). The service's IP address is never present in the descriptor, at any layer. The only sensitive information is the list of introduction point node_ids; this is mitigated by the 5-of-many selection strategy and epoch rotation.

7. Client-Side Lookup

A client wishing to connect to a .phantom address performs the following sequence before any connection attempt:

```
async fn lookup_hidden_service(address: &str) -> Result<ServiceDescriptor, HsError>
{
    // 1. Parse address
    let addr_bytes = parse_phantom_address(address)?;

    // 2. Derive DHT key for current epoch
    let epoch = current_epoch();
    let dht_key = blake3(
        b"phantom-hs-desc-v1" || &addr_bytes || &epoch.to_be_bytes()
    );

    // 3. Multi-path lookup (d=5, quorum=3 — same as node lookup, Spec № 3 §8)
    let raw = dht.lookup_phantom(Namespace::HiddenSvc, dht_key).await
        .ok_or(HsError::ServiceNotFound)?;

    // 4. Decode and verify
    let desc = ServiceDescriptor::decode(&raw)?;
    desc.verify_self_auth(&addr_bytes)?;      // address matches keys
    desc.verify_signatures()?;                // both Ed25519 + Dilithium-3
    desc.verify_epoch(epoch)?;                // not stale

    Ok(desc)
}
```

Listing 8 — Client hidden service lookup

If the current epoch descriptor is not found, the client retries with epoch - 1 (to handle clients whose clock is slightly ahead of the epoch boundary). If neither epoch is found, the service is offline or does not exist. There is no way to distinguish these cases by design — revealing "service exists but is offline" would leak information about the service's presence.

8. Rendezvous Protocol — Full Exchange

The rendezvous protocol establishes a shared session key between client and server without either party learning the other's IP address. The protocol involves five actors: the client, an introduction point (selected by the service), a rendezvous node (selected by the client), the Phantom mix network (routing Sphinx⁺ packets between all parties), and the hidden service itself.

8.1 Protocol Overview

```
RENDEZVOUS PROTOCOL OVERVIEW
================================

Phase 1: Client preparation
  Client selects a rendezvous node R from the DHT.
  Client builds a SURB (Single-Use Reply Block) addressed to R.
  Client sends a rendezvous request to the service via an introduction point.

Phase 2: Service response
  Service receives the rendezvous request at the introduction point.
  Service builds a Sphinx⁺ circuit to R.
  Service sends a rendezvous join message to R via its circuit.

Phase 3: Key exchange
  R forwards the service's join message to the client via the SURB.
  Client and service exchange hybrid KEM ciphertexts through R.
  Both derive the same session key. R sees only ciphertext.

Phase 4: Session
  All subsequent traffic is end-to-end encrypted.
  Routed through the mix network. R is no longer a bottleneck.
```

Listing 9 — Rendezvous protocol phase overview

8.2 Detailed Message Sequence

Step	Actor	Action	Visible to R	Visible to IP
1	Client	Selects rendezvous node R; generates ephemeral KEM keys	Nothing yet	Nothing yet
2	Client	Builds SURB for R → Client return path	SURB header (opaque)	Nothing
3	Client	Sends IntroRequest to IP via Sphinx⁺ circuit	Nothing	service_address, rendezvous_token, pk_client_kem, SURB
4	IP	Forwards IntroRequest to service via separate Sphinx⁺ path	Nothing	Acts as relay only

Step	Actor	Action	Visible to R	Visible to IP
5	Service	Receives IntroRequest; verifies PoW and signature	Nothing	Sees IntroRequest payload
6	Service	Builds Sphinx⁺ circuit to R; sends RendezvousJoin	service_address, pk_service_kem, rendezvous_token	Nothing
7	R	Matches rendezvous_token; forwards pk_service_kem to client via SURB	Encrypted tokens only	Nothing
8	Client	Computes session key from hybrid KEM; sends confirmation to R	Encrypted confirmation	Nothing
9	R	Forwards confirmation to service	Encrypted tokens only	Nothing
10	Both	Session established; R is no longer in the critical path	—	—

Table 3 — Rendezvous message sequence with per-actor visibility

8.3 IntroRequest Format

```
#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]
pub struct IntroRequest {
    pub service_address:   [u8; 32],
    pub rendezvous_token:  [u8; 32],   // random; matches service's circuit to R
    pub rendezvous_node_id:[u8; 32],   // R's node_id (service looks up R via DHT)
    pub pk_client_x25519:  [u8; 32],   // client's ephemeral X25519 KEM pubkey
    pub pk_client_kyber:   [u8; 1568], // client's ephemeral Kyber-1024 KEM pubkey
    pub surb:              SurbBlock,  // SURB for R → client return path
    pub pow_nonce:         [u8; 8],    // proof-of-work solution (see §12)
    pub timestamp:         u64,
    pub sig_ed_ephemeral:  [u8; 64],   // signed with client ephemeral key
}

// SURB: Single-Use Reply Block
// Contains a pre-built Sphinx⁺ return path from R back to the client.
// R cannot read the SURB's contents — it is opaque encrypted routing data.
// R uses it exactly once. Re-use is detectable (replay cache).
pub struct SurbBlock {
    pub alpha_cl:     [u8; 32],
    pub alpha_pq:     [u8; 96],
    pub beta_routing: [u8; 128],
    pub gamma_mac:    [u8; 32],
}
```

Listing 10 — IntroRequest and SURB formats

8.4 RendezvousJoin Format

```
#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]
pub struct RendezvousJoin {
    pub rendezvous_token:   [u8; 32],    // matches the IntroRequest token
    pub pk_service_x25519:  [u8; 32],    // service's epoch X25519 KEM pubkey
    pub ct_kyber:           [u8; 1568],  // Kyber-1024 encapsulation to client
pk_client_kyber
    pub timestamp:          u64,
    pub sig_ed:             [u8; 64],    // Ed25519(sk_ed, canonical_bytes)
    pub sig_dil:            [u8; 3293],  // Dilithium-3(sk_dil, canonical_bytes)
}
// Signed with permanent keys: client can verify service identity from the address.
```

Listing 11 — RendezvousJoin format

9. Cryptographic Key Exchange over the Rendezvous Channel

The rendezvous key exchange uses the same hybrid construction as the Sphinx⁺ per-hop key derivation (Technical Specification № 2), adapted for the end-to-end session context. Both X25519 and Kyber-1024 are computed independently; their shared secrets are combined with BLAKE3.

9.1 Client Key Derivation

```
// Client has: sk_client_x25519, sk_client_kyber (ephemeral, generated for this
session)
// Client has received RendezvousJoin containing:
//   pk_service_x25519  (service's epoch key)
//   ct_kyber           (Kyber-1024 ciphertext encapsulated to client's pubkey)

// Step 1: Classical shared secret
let ss_classical = X25519(sk_client_x25519, pk_service_x25519);

// Step 2: Post-quantum shared secret (client decapsulates)
let ss_pq = Kyber1024::decaps(sk_client_kyber, ct_kyber);

// Step 3: Derive session key (same combiner as Sphinx⁺ §3.2.4)
let session_key = blake3(
    b"phantom-hs-session-v1"
    || ss_classical
    || ss_pq
    || rendezvous_token   // domain separation per-session
    || service_address    // binds key to this service
    || epoch.to_be_bytes()
);
```

Listing 12 — Client-side session key derivation

9.2 Service Key Derivation

```
// Service has: sk_x25519 (epoch), sk_kyber (epoch)
// Service has received IntroRequest containing:
//   pk_client_x25519   (client's ephemeral key)
//   pk_client_kyber    (client's ephemeral Kyber key)

// Step 1: Classical shared secret (identical to client's ss_classical)
let ss_classical = X25519(sk_x25519, pk_client_x25519);

// Step 2: Post-quantum (service encapsulates to client's Kyber pubkey)
let (ct_kyber, ss_pq) = Kyber1024::encaps(pk_client_kyber);
// ct_kyber is sent in RendezvousJoin; ss_pq is used locally

// Step 3: Same combiner — produces same session_key as client
let session_key = blake3(
    b"phantom-hs-session-v1"
    || ss_classical
    || ss_pq
    || rendezvous_token
    || service_address
    || epoch.to_be_bytes()
);
```

Listing 13 — Service-side session key derivation

9.3 Security Properties of the Key Exchange

Property	Guarantee	Basis
Forward secrecy	Compromise of epoch KEM keys does not expose sessions from prior epochs	Ephemeral X25519 + per-epoch Kyber keys deleted after epoch end
Post-quantum security	Session key secure against quantum adversary	Kyber-1024 encapsulation provides IND-CCA2 security; session key requires both to break
Service authentication	Client verifies it is talking to the correct service	RendezvousJoin is signed with permanent Ed25519 + Dilithium-3 keys bound to service_address
Client anonymity at service	Service cannot identify client across sessions	Client uses fresh ephemeral keys per session; no persistent client identity
Rendezvous node isolation	R learns neither the session key nor either party's identity	R sees only ciphertexts; key derivation is end-to-end

Table 4 — Security properties of the rendezvous key exchange

10. Session Establishment and Ongoing Communication

10.1 Post-Rendezvous Circuit

Once the session key is established through the rendezvous node, the rendezvous node is no longer required. The client and service each independently build fresh Sphinx⁺ circuits and communicate end-to-end through the mix network, with the session key used to authenticate and encrypt application-layer data on top of the Sphinx⁺ transport encryption.

```
// Client-side post-rendezvous session
pub struct HsSession {
    pub service_address:  [u8; 32],
    pub session_key:      [u8; 32],
    pub epoch:            u32,
    pub send_circuit:     SphinxCircuit,   // client → mix network → service
    pub recv_circuit:     SphinxCircuit,   // service → mix network → client
    pub seq_send:         u64,             // replay protection
    pub seq_recv:         u64,
}

// All application data is ChaCha20-Poly1305 encrypted under session_key
// before being placed into Sphinx⁺ packet payloads.
// The session key never leaves the two endpoints.
```

Listing 14 — Post-rendezvous session structure

10.2 Application Layer Integration

Phantom Hidden Services expose a stream-oriented interface to applications, abstracting the underlying Sphinx⁺ packet routing. Applications use the hidden service API identically to a standard TCP socket, with the difference that the remote address is a .phantom string rather than an IP address and port.

```
// Application-level hidden service API
pub trait PhantomListener {
    /// Accept an incoming connection from any client.
    async fn accept(&self) -> Result<HsStream, HsError>;
}

pub trait PhantomDialer {
    /// Connect to a .phantom address.
    async fn connect(&self, address: &str) -> Result<HsStream, HsError>;
}

pub struct HsStream {
    // Implements AsyncRead + AsyncWrite (tokio)
    // Internally: chunks application data into Sphinx⁺ packet payloads
    // Internally: reassembles received packets into ordered byte stream
}
```

Listing 15 — Application-layer hidden service API

11. Epoch Rotation and Forward Secrecy

11.1 Rotation Timeline

```
T - 900s:  Service generates new KEM keypair (X25519 + Kyber-1024) for next epoch.
           Begins re-registering with introduction points for next epoch.

T - 300s:  Service publishes next-epoch ServiceDescriptor to DHT.
           Both current and next descriptors valid during overlap period.

T +   0s:  New epoch begins. Current-epoch descriptor still valid for 120s.

T + 120s:  Previous-epoch descriptor purged from DHT.
           Service deletes previous-epoch sk_x25519 and sk_kyber.
           Sessions started in the previous epoch continue (session_key already
derived).
           New sessions must use the new epoch's descriptor.

T +3600s:  Next epoch — cycle repeats.
```

Listing 16 — Hidden service epoch rotation timeline

11.2 Forward Secrecy Guarantee

Deleting previous-epoch KEM private keys ensures that an adversary who compromises the service's current key material cannot decrypt sessions established in previous epochs. The session_key for each session is derived from ephemeral key material; once that key material is deleted, the session_key cannot be rederived even with full access to the service's current state.

Forward Secrecy Scope

Protected: Session keys for all sessions whose key exchange completed before the epoch boundary. Deleting epoch KEM keys prevents rederivation.

Not protected: Active sessions whose session_key is currently in memory. Physical compromise of the running process exposes current session keys. This is a fundamental property of any in-memory session key, not a Phantom-specific limitation.

Signing keys: Permanent signing keys (Ed25519, Dilithium-3) do not rotate. Compromise allows impersonation going forward, but does not retroactively expose past session keys.

12. Denial-of-Service Resistance

Hidden services face a unique DoS threat: an adversary can flood introduction points with illegitimate contact requests, consuming the service's resources processing them and potentially revealing the service's circuit-building behaviour to traffic analysis. Phantom Protocol addresses this with a proof-of-work puzzle that every client must solve before an IntroRequest is forwarded to the service.

12.1 Introduction-Layer PoW Puzzle

```
// Client must solve before sending IntroRequest:
// Find nonce such that BLAKE3(service_address || rendezvous_token || epoch ||
nonce)
//   has at least pow_bits leading zero bits.
//
// pow_bits is published in the ServiceDescriptor (field: intro_pow_bits).
// Default: 18 bits (~262,144 hash evaluations; ~8ms on modern hardware)
// Under attack: service increments pow_bits in next descriptor (max: 28 bits)

fn solve_intro_pow(service_address: &[u8; 32], rendezvous_token: &[u8; 32],
                   epoch: u32, pow_bits: u8) -> [u8; 8] {
    let mut nonce = [0u8; 8];
    loop {
        let hash = blake3::hash(
            &[service_address, rendezvous_token, &epoch.to_be_bytes(),
&nonce].concat()
        );
        if hash.leading_zeros() >= pow_bits as u32 { return nonce; }
        increment_nonce(&mut nonce);
    }
}

// Verification at introduction point (fast, no DHT lookup needed):
fn verify_intro_pow(req: &IntroRequest, pow_bits: u8) -> bool {
    let hash = blake3::hash(
        &[&req.service_address, &req.rendezvous_token,
          &req.timestamp_epoch().to_be_bytes(), &req.pow_nonce].concat()
    );
    hash.leading_zeros() >= pow_bits as u32
}
```

Listing 17 — Introduction-layer PoW puzzle

12.2 Adaptive Difficulty

```
const DEFAULT_POW_BITS: u8 = 18;
const MAX_POW_BITS: u8 = 28;
const INTRO_RATE_THRESHOLD: u32 = 1_000;  // requests/minute triggers increase

fn adjust_pow_difficulty(current_bits: u8, request_rate: u32) -> u8 {
    if request_rate > INTRO_RATE_THRESHOLD {
        (current_bits + 1).min(MAX_POW_BITS)
    } else if request_rate < INTRO_RATE_THRESHOLD / 10 {
        current_bits.saturating_sub(1).max(DEFAULT_POW_BITS)
    } else {
        current_bits
    }
```

```
}
// Difficulty is published in the next epoch's ServiceDescriptor.
```

Listing 18 — Adaptive PoW difficulty

12.3 Additional DoS Mitigations

- Introduction points enforce a per-service rate limit of MAX_INTRO_REQUESTS requests per minute (default: 5,000). Excess requests are dropped without processing.
- Introduction points do not forward requests with invalid PoW solutions. The PoW verification is O(1) and performed before any circuit-building or DHT operations.
- Rendezvous tokens are single-use. An introduction point that receives a duplicate rendezvous_token drops the request silently. This prevents replays of valid IntroRequests.
- Service descriptors do not include the service's onion address or any information enabling reflection attacks. The introduction point's only role is to forward; it holds no information that could be abused for amplification.

13. Rust Implementation Guide

13.1 Crate Dependencies

Component	Crate	Notes
Ed25519 signatures	ed25519-dalek	Signing and verification; same as DPKI layer
Dilithium-3 signatures	pqcrypto-dilithium	PQ signing; same as DPKI layer
X25519 KEM	x25519-dalek	Classical KEM; same as Sphinx⁺ layer
Kyber-1024 KEM	pqcrypto-kyber	PQ KEM; same as Sphinx⁺ layer
BLAKE3 hashing	blake3	All hashing and key derivation
ChaCha20-Poly1305	chacha20poly1305	Session-level encryption (RustCrypto)
DHT operations	phantom-dpki	Internal crate from Technical Spec № 3
Sphinx⁺ packets	phantom-sphinx-plus	Internal crate from Technical Spec № 2
Async runtime	tokio	Async I/O for all network operations
Serialisation	serde + postcard	No-std compatible binary serialisation
Zeroisation	zeroize	Epoch KEM secret key deletion on rotation
Base32 encoding	base32	Address encoding/decoding

Table 5 — Rust crate dependencies

13.2 Module Structure

```
phantom-hidden-service/
├── src/
│   ├── address.rs          // .phantom address derivation and verification
│   ├── keys.rs             // Key generation, epoch rotation, key deletion
│   ├── descriptor.rs       // ServiceDescriptor struct, signing, DHT publication
│   ├── intro/
│   │   ├── mod.rs          // Introduction point selection and registration
│   │   ├── registration.rs// IntroRegistration message format and handling
│   │   └── dos.rs          // PoW puzzle and rate limiting
│   ├── rendezvous/
│   │   ├── mod.rs          // Rendezvous protocol orchestration
│   │   ├── request.rs      // IntroRequest construction and SURB generation
│   │   ├── join.rs         // RendezvousJoin handling
│   │   └── kex.rs          // Hybrid key exchange (X25519 + Kyber-1024)
│   ├── session.rs          // HsSession, post-rendezvous communication
│   ├── stream.rs           // HsStream: AsyncRead + AsyncWrite impl
│   └── lib.rs              // Public API
├── tests/
│   ├── integration/        // Multi-node tests (client + service + intros)
│   └── unit/
└── benches/                // Criterion benchmarks: key exchange, PoW, SURB gen
```

Listing 19 — Module structure

13.3 Core API

```
pub struct PhantomHiddenService {
    address:     String,          // e.g. "3xk7m2nq...aaaa.phantom"
    descriptor:  ServiceDescriptor,
    intro_nodes: Vec<IntroState>,
    session_mgr: SessionManager,
}

impl PhantomHiddenService {
    /// Start a hidden service. Generates keys, registers intro points, publishes
descriptor.
    pub async fn start(config: HsConfig) -> Result<Self, HsError>;

    /// Accept an incoming client connection.
    pub async fn accept(&self) -> Result<HsStream, HsError>;

    /// Returns the .phantom address for sharing with clients.
    pub fn address(&self) -> &str;

    /// Connect to a .phantom address as a client.
    pub async fn connect(address: &str) -> Result<HsStream, HsError>;

    /// Gracefully stop the hidden service (rotates away, does not announce
offline).
    pub async fn stop(self) -> Result<(), HsError>;
}

/// Configuration for a hidden service
pub struct HsConfig {
    pub key_dir:          PathBuf,     // directory holding long-term signing keys
    pub n_intro:          usize,       // default: 5
    pub intro_pow_bits:   u8,          // default: 18
    pub dpki:             Arc<PhantomDPKI>,  // from Spec № 3
}
```

Listing 20 — PhantomHiddenService public API

14. Attack Analysis

Attack	Mechanism	Phantom HS Resistance	Residual Risk
Introduction point enumeration	Enumerate all Phantom nodes to identify which serve as introduction points	Introduction points are not distinguishable from regular Phantom nodes by traffic alone; they do not announce their role publicly	A global passive adversary correlating registration circuits may identify introduction points; mitigated by multiple independent intro points and epoch rotation
Introduction point compromise	Compromise all N_INTRO introduction points to prevent client contact	AS diversity in selection; 5 independent nodes required; new intro points selected each epoch	State-level adversary compromising 5 ASes simultaneously; unlikely but not impossible against a high-value target
Traffic correlation via rendezvous node	Control the rendezvous node to correlate client and service traffic	ZK shuffle proof on all rendezvous path traffic prevents ordering attacks; rendezvous node rotated per-session	Compromised rendezvous node can observe traffic volume and timing; does not learn identity of either party
Service enumeration via DHT	Enumerate DHT to find all hidden service descriptors	Descriptors are published under BLAKE3(address || epoch); brute-forcing is computationally infeasible for addresses longer than ~40 bits	Targeted search for a known address is trivial — descriptors are public by design; anonymity relies on the address not being linked to a real-world identity
SURB replay attack	Replay a captured SURB to forward a forged message to the client	SURBs are single-use; introduction points maintain a replay cache keyed on the SURB's alpha_cl field; duplicate alpha_cl is rejected	None — mathematically prevented by the replay cache
DoS via introduction flooding	Flood introduction points with requests to exhaust service resources	PoW puzzle with adaptive difficulty; per-service rate limit at introduction points	Adversary with significant compute can sustain attack at higher PoW difficulties; difficulty cap (28 bits) limits legitimate client usability as a side effect
Quantum key recovery from address	Use quantum computer to derive service private key from public address	Address derivation includes Dilithium-3 and Kyber-1024 public keys; hybrid construction requires breaking both Ed25519/X25519 AND	None at current NIST PQ security levels

Attack	Mechanism	Phantom HS Resistance	Residual Risk
		Dilithium-3/Kyber-1024 simultaneously	
Epoch correlation	Correlate service activity across epoch boundaries to track service despite key rotation	DHT keys change each epoch; introduction points change each epoch; service address is permanent but must be known in advance	An adversary who already knows the service address can correlate across epochs; epoch rotation limits the value of compromised KEM keys, not of the address itself

Table 6 — Attack analysis for Phantom Hidden Services

15. Open Problems

Phantom Hidden Services are honest about the problems not yet solved:

15.1 Introduction Point Anonymity Set

The current design selects N_INTRO = 5 introduction points per service per epoch. A global passive adversary who can observe all Phantom traffic can attempt to correlate introduction-point registration circuits with the service's exit point to identify the service's IP. Increasing N_INTRO reduces this risk at the cost of higher overhead. Research into verifiable secret sharing across introduction points — where no single introduction point holds enough state to forward a connection without cooperation of a threshold of others — is a potential future direction.

15.2 Onion-Address Discovery and Sharing

.phantom addresses are long (60 characters) and meaningless to humans. There is no Phantom-native naming system analogous to DNS for mapping human-readable names to .phantom addresses. A naming system that is itself decentralised, censorship-resistant, and does not require a trusted registry is an open design problem. Existing approaches (Namecoin, ENS) introduce their own trust assumptions.

15.3 Persistent Hidden Services under Churn

If a hidden service goes offline for longer than one epoch, its descriptor expires from the DHT. Clients who attempt to connect during this window receive a not-found response indistinguishable from a service that has permanently ceased operating. A lightweight 'I am temporarily offline' signal that does not reveal the service's identity or location is an open design problem.

15.4 Client Anonymity Against the Service

In the current design, the service cannot learn the client's IP address. However, if the service is malicious, it can attempt to correlate session timing and traffic patterns with network-level observations to deanonymize clients. The ZK shuffle proof on the mix network path mitigates this by enforcing verifiable mixing between the client and the service. The residual risk is a powerful adversary controlling both the hidden service and a substantial fraction of the Phantom network simultaneously.

15.5 AS Diversity Enforcement

The AS diversity requirement for introduction point selection relies on accurate IP-to-ASN mapping data. This data is publicly available (CAIDA, RouteViews) but must be kept current. Stale ASN data could result in multiple introduction points at the same AS being selected, undermining the diversity guarantee. A mechanism for in-network ASN data distribution is under consideration.

16. References

[DG09] Danezis, G., Goldberg, I. — Sphinx: A Compact and Provably Secure Mix Format. IEEE S&P 2009.

[DKRMM04] Dingledine, R., Mathewson, N., Syverson, P. — Tor: The Second-Generation Onion Router. USENIX Security 2004.

[Tor-HS] Dingledine, R., Mathewson, N. — Tor Hidden Service Protocol. Tor Design Document. https://spec.torproject.org/rend-spec-v3

[SFG+07] Shmatikov, V., Wang, M.-H. — Timing Analysis in Low-Latency Mix Networks. ESORICS 2006.

[NIST203] NIST — CRYSTALS-Kyber (ML-KEM), FIPS 203. 2024.

[NIST204] NIST — CRYSTALS-Dilithium (ML-DSA), FIPS 204. 2024.

[BM07] Baumgart, I., Mies, S. — S/Kademlia: A Practicable Approach Towards Secure Key-Based Routing. IEEE ICPADS 2007.

[PHS3] Phantom Protocol — Technical Specification № 3: DHT-Based Trustless PKI. March 2026.

[PHS2] Phantom Protocol — Technical Specification № 2: Sphinx^{+} and ZK Verifiable Shuffle. March 2026.

[PWPW] Phantom Protocol — Whitepaper v0.1. March 2026.

[RFC4648] Josefsson, S. — The Base16, Base32, and Base64 Data Encodings. IETF RFC 4648. 2006.

— Phantom Protocol Technical Specification № 1 — End of Document —